

Studi'OS

Roadmap - Desktop Distribution, Updates & Public Registration

Objectif : rendre Studi'OS distribuable au grand public, avec un client Desktop maintenable et auto-mis à jour, tout en permettant la création autonome et sécurisée d'un compte utilisateur.

Périmètre	Distribution Desktop, cycle de mise à jour, inscription publique et cycle de vie du compte
Principe	Une seule codebase Dashboard ; Web et Desktop restent deux modes de distribution
Cible	Windows en priorité ; architecture extensible à d'autres OS
Dépendance	À démarrer après stabilisation/validation du chantier Desktop actuel
Inscription publique	Création autonome d'un compte sur une instance Studi'OS qui l'autorise
Hors périmètre	Multi-tenant SaaS complet, billing, marketplace, hébergement d'IA, refonte métier générale

Résultat attendu : un nouvel utilisateur peut découvrir Studi'OS, créer et vérifier son compte, se connecter depuis le Web ou le Desktop, puis recevoir les mises à jour Desktop sans intervention manuelle. L'inscription publique ne doit pas transformer implicitement Studi'OS en SaaS multi-tenant.

1. Principes d'architecture

- Source unique. Le répertoire Dashboard reste la source du frontend ; aucun fork durable « dashboard-web » / « dashboard-desktop ».
- Deux distributions. Le Web est servi par l'infrastructure Studi'OS ; le Desktop embarque le même frontend dans Tauri avec les capacités locales nécessaires.
- Inscription contrôlée par l'instance. Une instance peut activer ou désactiver l'inscription publique. L'activation ne crée pas à elle seule une architecture multi-tenant.
- Identité unique. Le même compte doit fonctionner depuis le Web et le Desktop ; les capacités locales du Desktop restent distinctes de l'identité serveur.
- Least privilege. Un compte créé publiquement reçoit un rôle minimal explicitement défini ; aucun rôle admin/developer ne doit être obtenu par auto-inscription.
- Compatibilité avant synchronisation. Le serveur conserve une fenêtre de compatibilité avec les versions Desktop anciennes.
- Contrats protégés. Les contrats HTTP et studio.local/v1 évoluent de manière additive tant qu'aucune version majeure n'est engagée.

Flux cible

Découvert e	Inscription	Vérification	Connexion	Onboardin g	Usage	Release	Auto-updat e
----------------	-------------	--------------	-----------	----------------	-------	---------	-----------------

2. Règles de compatibilité proposées

Type de changement	Rebuild Desktop	Ancienne version
Backend interne, contrat inchangé	Non	Compatible
Nouvel endpoint non utilisé	Non	Compatible
Dashboard React/CSS/TS	Oui pour recevoir l'UI	Compatible mais ancienne
Tauri / Rust / sidecar / capacités locales	Oui	Selon changement
Ajout auth rétrocompatible	Selon usage	Compatible
Rupture API / auth / studio.local/v1	Oui	Potentiellement incompatible

DU-0 - Architecture Gate & audit

Objectif : Établir l'état réel avant modification et intégrer l'authentification publique au cadrage.

- Cartographier Dashboard -> Tauri -> sidecar -> serveur et les flux d'authentification existants.
- Auditer provisioning actuel, rôles, JWT/MachineBearer, création d'utilisateurs et bootstrap admin.
- Identifier les artefacts embarqués, leur versionnement et le mécanisme de release existant.
- Décider où vit la politique d'inscription publique et comment une instance l'active/désactive.
- Définir les changements nécessitant un rebuild Desktop et ceux purement serveur.

Gate : décision d'architecture approuvée avant implémentation.

DU-1 - Modèle d'inscription publique & cycle de compte

Objectif : Permettre l'auto-crédation d'un compte sans affaiblir le modèle de rôles existant.

- Définir le rôle initial autorisé pour un compte auto-inscrit et les transitions nécessitant une action admin.
- Définir états du compte : pending/unverified, active, disabled, éventuellement locked.
- Ajouter inscription, vérification d'adresse e-mail et renvoi de vérification.
- Prévoir mot de passe oublié/réinitialisation et changement de mot de passe authentifié.
- Préserver le bootstrap du premier admin et le provisioning administratif existant.
- Définir suppression/désactivation de compte sans introduire de billing ni de multi-tenant implicite.

Gate : un utilisateur peut créer, vérifier, récupérer et utiliser un compte sans pouvoir s'auto-élever en privilèges.

DU-2 - Sécurité anti-abus & secrets d'authentification

Objectif : Rendre l'exposition publique acceptable avant ouverture Internet.

- Rate limiting sur inscription, connexion, vérification et récupération.
- Protection contre énumération de comptes dans les réponses publiques.
- Politique de mots de passe et stockage via le mécanisme de hashage existant/audité.
- Tokens de vérification/récupération courts, expirables, à usage unique et non journalisés en clair.
- Journaliser les événements de sécurité utiles sans exposer credentials, JWT ou secrets.
- Définir stratégie CAPTCHA/challenge seulement si l'abus réel le justifie.

Gate : tests de sécurité et d'abus couvrent les chemins publics critiques.

DU-3 - Onboarding Web & Desktop

Objectif : Offrir un parcours cohérent après installation ou première visite.

- Écran « créer un compte / se connecter » partagé par le Dashboard.
- Le Desktop permet de choisir/configurer l'instance Studi'OS puis utilise le même flux d'identité.
- Conserver les credentials/tokens dans un stockage local adapté et ne pas les inclure dans les logs/export diagnostics.
- Gérer vérification e-mail en attente, session expirée, serveur inaccessible et compte désactivé.
- Distinguer clairement compte serveur, machine enregistrée et agent.

Gate : parcours installation -> inscription -> connexion -> Dashboard fonctionnel sur Web et Desktop.

DU-4 - Versioning & politique de compatibilité

Objectif : Définir une politique simple et vérifiable pour le Desktop et les contrats d'auth.

- Adopter SemVer pour le Desktop.
- Définir current/latest/minimum_supported côté serveur ou manifeste de release.
- Définir la fenêtre de compatibilité des anciennes versions.
- Interdire les ruptures silencieuses des contrats v1 et des flux d'auth consommés par le Desktop.
- Définir le comportement d'un client trop ancien.

Gate : matrice de compatibilité documentée et testable.

DU-5 - Build Desktop reproductible

Objectif : Pouvoir reconstruire une release depuis Git.

- Build Dashboard déterministe.
- Build du sidecar/runtime local.
- Build Tauri Windows depuis un tag/commit.
- Version injectée de façon unique dans les composants concernés.
- Hashes des artefacts et provenance de build.

Gate : deux builds du même tag produisent un résultat fonctionnel équivalent.

DU-6 - Signature & confiance

Objectif : Empêcher l'installation d'une mise à jour non authentique.

- Mettre en place la signature attendue par l'updater Tauri.
- Séparer clés de signature et dépôt.
- Prévoir la signature de code Windows pour réduire les alertes SmartScreen.
- Documenter rotation/révocation des clés.
- Ne jamais placer de clé privée dans Git ou dans le client.

Gate : une mise à jour altérée ou non signée est refusée.

DU-7 - Pipeline de release CI/CD

Objectif : Automatiser la production d'une release.

- Déclenchement contrôlé par tag/release.
- Exécuter tests backend/contrats/auth/Dashboard/Desktop pertinents.
- Construire Dashboard, sidecar puis Tauri.
- Signer les artefacts et générer le manifeste d'update.
- Publier les artefacts dans le canal choisi et conserver logs, hashes et version source.

Gate : une release candidate peut être produite sans manipulation locale ad hoc.

DU-8 - Auto-update Desktop

Objectif : Synchroniser réellement les installations utilisateur.

- Vérifier au démarrage ou périodiquement la disponibilité d'une version.
- Télécharger via HTTPS.
- Vérifier signature/intégrité avant installation.
- Proposer installation/redémarrage avec UX claire.
- Gérer réseau indisponible sans bloquer le produit.
- Ne jamais détruire configuration, session ou données locales lors de l'update.

Gate : N peut installer N+1 depuis l'application sans réinstallation manuelle.

DU-9 - Compatibilité serveur / Desktop / Auth

Objectif : Éviter qu'un déploiement serveur casse les clients ou leur connexion.

- Exposer/distribuer la version minimale supportée.
- Distinguer update recommandée et obligatoire.
- Afficher une erreur explicite si le client est réellement incompatible.
- Prévoir une période de grâce pour les changements compatibles.
- Tester les contrats consommés par N-1 et N, y compris login/session/refresh si applicable.

Gate : le serveur distingue client ancien compatible et client réellement incompatible.

DU-10 - Tests E2E inscription + upgrade N-1 -> N

Objectif : Valider le parcours réel d'un nouvel utilisateur et la migration entre releases.

- Tester inscription publique depuis zéro, vérification e-mail, login et récupération de compte.
- Installer N-1 sur environnement propre puis configurer compte, serveur, workspace et données locales.
- Mettre à jour vers N et vérifier session/credentials, paramètres, workspace, Knowledge/Graphify et fonctions locales.
- Tester interruption réseau, update échouée, token expiré et compte désactivé.
- Vérifier qu'aucune élévation de rôle n'est possible par les endpoints publics.

Gate : inscription complète et scénario N-1 -> N démontrés de bout en bout.

DU-11 - Canaux de release & ouverture progressive

Objectif : Réduire le risque avant diffusion générale.

- Canal stable pour utilisateurs normaux et beta/preview optionnel.
- Pas de divergence durable de code entre canaux.
- Promotion d'un même artefact validé lorsque possible.
- Pouvoir ouvrir l'inscription publique d'abord sur un environnement/groupe contrôlé.
- Documenter retour vers stable et fermeture temporaire des nouvelles inscriptions sans bloquer les comptes existants.

Gate : une version et l'inscription publique peuvent être validées progressivement avant ouverture générale.

DU-12 - Gate de production publique

Objectif : Prouver le cycle complet sur une instance exposée dans des conditions réalistes.

- Installation Desktop depuis zéro et accès Web.
- Création + vérification d'un nouveau compte.
- Connexion à une instance Studi'OS réelle.
- Utilisation Dashboard + fonctions locales.
- Publication d'une version suivante puis détection/téléchargement/installation/redémarrage.
- Vérification de l'intégrité des données et de la session après update.
- Runbook incident : suspendre release, désactiver inscriptions, révoquer session/compte, rollback.

Gate : cycle découverte -> inscription -> usage -> update -> reprise validé de bout en bout.

3. Contraintes à préserver

- Une seule source de code pour le Dashboard.
- Aucun modèle IA ou agent hébergé sur le VPS par cette roadmap.
- L'inscription publique est configurable par instance et ne donne jamais de privilège administratif par défaut.
- L'inscription publique ne signifie pas automatiquement multi-tenant SaaS : isolation entre organisations, billing et plans restent hors périmètre tant qu'une décision dédiée ne les introduit pas.
- Le Desktop conserve ses capacités locales et son comportement offline/reconnect.
- Secrets de signature, secrets e-mail et tokens d'authentification ne sont jamais commités ni exposés dans les diagnostics.
- Une mise à jour échouée ne détruit pas l'installation fonctionnelle précédente.
- Les contrats existants restent rétrocompatibles sauf décision explicite de version majeure.
- Le pipeline de release ne contourne pas les tests/gates existants.

4. Livrables attendus

Livrable	Attendu
Décision d'architecture	Source unique, inscription publique, stratégie d'update, compatibilité, canaux
Contrat d'inscription	États de compte, rôle initial, vérification, récupération, désactivation
Sécurité publique	Rate limits, anti-énumération, gestion des tokens/secrets, audit
Onboarding	Parcours Web/Desktop cohérent jusqu'au premier usage
Politique de version	SemVer + minimum supporté + règles de rupture
Pipeline CI/CD	Build, tests, signature, publication
Auto-updater	Détection, téléchargement, vérification, installation
Tests E2E	Inscription propre + installation propre + N-1 -> N
Runbook production	Publier, ouvrir/fermer inscriptions, suspendre, révoquer, récupérer une release

5. Ordre recommandé

DU-0 doit cadrer l'architecture. DU-1 à DU-3 sécurisent l'ouverture publique et le parcours utilisateur. DU-4 à DU-9 industrialisent la distribution et la compatibilité. DU-10 à DU-12 valident le tout dans des conditions proches de la production.

Le chantier d'inscription publique peut avancer en parallèle d'une partie de la distribution Desktop après DU-0, mais le gate final ne doit être validé que lorsque les deux parcours se rejoignent réellement.

6. Consigne pour proposition dans Studi'OS

Claude doit utiliser les outils MCP Studi'OS pour proposer cette roadmap dans le projet approprié. La proposition doit conserver les phases DU-0 à DU-12, leurs objectifs, gates et dépendances. Elle peut être enrichie après audit du dépôt, mais tout changement important de périmètre doit être explicité.

Important : ce document reste un cadrage. Sa présence dans Studi'OS ne constitue pas une autorisation d'implémenter immédiatement les phases. La

roadmap doit d'abord être proposée via MCP, reliée aux dépendances utiles puis passer par le processus de review/approbation.

Dépendance recommandée : lancer DU-0 lorsque la roadmap Desktop actuelle est suffisamment stabilisée pour auditer l'architecture cible. L'inscription publique ne doit être ouverte sur Internet qu'après validation des gates DU-1, DU-2 et DU-3.